

Program 1:

```
#include <stdio.h>

int main(){
int x = 10;

printf("Value of x = %d\n", x);
printf("Virtual address of x = %p\n", (void *)&x);

return 0;
}
```

Output:

```
crazyguy64@CrazyGuy64:~/skill7$ nano program1.c
crazyguy64@CrazyGuy64:~/skill7$ gcc program1.c -o program1
crazyguy64@CrazyGuy64:~/skill7$ ./program1
Value of x = 10
Virtual address of x = 0x7ffee8273274
```

Program 2:

```
#include<stdio.h>
#include <stdlib.h>

int global = 10;
int bss_variable;

int main(){
int local = 10;

int *heap = malloc(sizeof(int));
*heap = 30;

printf("Code address : %p\n", (void *)main);
printf("Data address : %p\n", (void *)&global);
printf("BSS address : %p\n", (void *)&bss_variable);
printf("Heap address : %p\n", (void *)heap);
printf("Stack address: %p\n", (void *)&local);

free(heap);

return 0;
}
```

Output:

```
crazyguy64@CrazyGuy64:~/skill7$ nano program2.c
crazyguy64@CrazyGuy64:~/skill7$ gcc program2.c -o program2
crazyguy64@CrazyGuy64:~/skill7$ ./program2
Code address : 0x5639ce7f61a9
Data address : 0x5639ce7f9010
BSS address : 0x5639ce7f9018
Heap address : 0x5639e06e3010
Stack address: 0x7ffd7cee6b1c
```

Program 3:

```
#include <stdio.h>

int main(){
int page_table[4] = {1, -1, 3, -1};
int page;

printf("Enter page number (0-3): ");
scanf("%d", &page);

if(page_table[page] == -1){
printf("Page Fault!\n");
printf("Page must be loaded into memory.\n");
}else{
printf("Page is already in memory. \n");
printf("Physical frame = %d\n", page_table[page]);
}

return 0;
}
```

Output:

```
crazyguy64@CrazyGuy64:~/skill7$ nano program3.c
crazyguy64@CrazyGuy64:~/skill7$ gcc program3.c -o program3
crazyguy64@CrazyGuy64:~/skill7$ ./program3
Enter page number (0-3): 2
Page is already in memory.
Physical frame = 3
```

Program 4:

```
#include <stdio.h>

int main(){
int page_table[] = {3, 5, 1, 7};
int page_size = 100;

int virtual_address = 250;

int page = virtual_address / page_size;
int offset = virtual_address % page_size;

int frame = page_table[page];

int physical_address = frame * page_size + offset;

printf("Virtual Address = %d\n", virtual_address);
printf("Page Number      = %d\n", page);
printf("Offset          = %d\n", offset);
printf("Frame Number     = %d\n", frame);
printf("Physical Address= %d\n", physical_address);

return 0;
}
```

Output:

```
crazyguy64@CrazyGuy64:~/skill7$ nano program4.c
crazyguy64@CrazyGuy64:~/skill7$ gcc program4.c -o program4
crazyguy64@CrazyGuy64:~/skill7$ ./program4
Virtual Address = 250
Page Number      = 2
Offset          = 50
Frame Number     = 1
Physical Address= 150
```

Program 5:

```
#include <stdio.h>
#include <stdlib.h>

int main(){
    int *p;

    p = malloc(5 * sizeof(int));

    if(p == NULL){
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++){
        p[i] = (i + 1) * 10;
    }

    for(int i = 0; i < 5; i++){
        printf("%d ", p[i]);
    }

    printf("\n");

    free(p);

    return 0;
}
```

Output:

```
crazyguy64@CrazyGuy64:~/skill7$ nano program5.c
crazyguy64@CrazyGuy64:~/skill7$ gcc program5.c -o program5
crazyguy64@CrazyGuy64:~/skill7$ ./program5
10 20 30 40 50
```

Program 6:

```
#include <stdio.h>

int main(){
int *p = NULL;

printf("Before accessing memory\n");

*p = 100;

printf("After accessing memory\n");

return 0;
}
```

Output:

```
crazyguy64@CrazyGuy64:~/skill7$ nano program6.c
crazyguy64@CrazyGuy64:~/skill7$ gcc program6.c -o program6
crazyguy64@CrazyGuy64:~/skill7$ ./program6
Before accessing memory
Segmentation fault ./program6
```

Program 7: